Using Design Patterns

6.0 Introduction to Patterns

This is the main body text in 12pt font.

The heading numbering stays consistent across the document.

6.1 Introduction

This is the main body text in 12pt font.

The heading numbering stays consistent across the document.

6.2 Abstraction-Occurence Pattern

6.2.0 Library Example

A Library book has properties such as:

- Title Duplicates per instance
- Author
- Barcode Unique per instance

Each instance of the same book will have a different barcode, but title and author is the same.

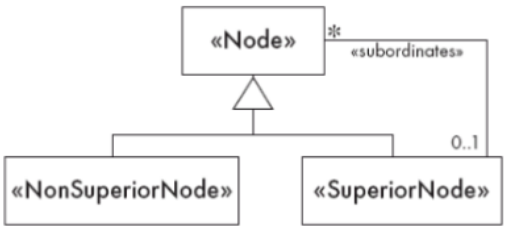
How can we make a mechanism that allows us to contain unique data instead of duplicate values?

The solution lies in a one-to-many association. A single "book" instance will have many "bookItem" instances, containing a unique barcode.

6.3 General Hiearchy Pattern

- Context: Objects in a hierarchy can have one or more objects above them (superiors), and one or more objects below them (subordinates). Some objects cannot have any subordinates
- Problem: How do you represent a hierarchy of objects, in which some objects cannot have subordinates?

- Forces: You want a flexible way of representing the hierarchy that prevents certain objects from having subordinates. All the objects have many common properties and operation
- Solution:



6.4 Player-Role Pattern

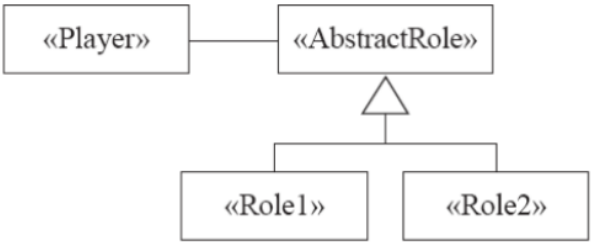
- Context: A role is a particular set of properties associated with an object within context. An object may play different roles in different contexts.
- Problem: What is the best way to model players and roles s.t a player can change roles or have multiple roles?
- Forces:

  Improve encapsulation by capturing role info

  Avoid multiple inheritance

  Instances cannot change classes

- Solution:

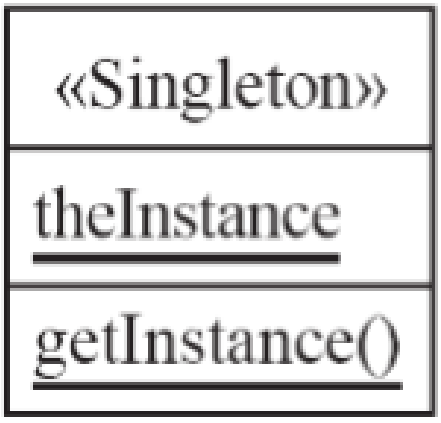


6.5 Singleton Pattern

- Context: We need classes that only have one instance
- Problem: How do we ensure the impossiblity of more than one instance of a singleton class
- Forces:

  Public constructors cannot ensure a single instance

  Singleton instance must also be accessible
- Solution: On instance, create a constructor only ONCE. Every constructor call past this will return the existing instance.



6.6 Observer Pattern

- Context:

  Associations between classes causes classes code to be inseparable.

  Reuse of one of these classes requires the other.
- Problem: How do we reduce the interconnection between classes?
- Forces: Maximize flexibility of the system.
- Solution:

6.7 Delegation Pattern

- Context: Another class has a method which provides a service we want to use
- Problem: How do we effectively make use of a method that already exists in the other class?
- Forces: Minimize development costs via reuse

6.8 Facade

- Context: Applications contain several complex packages. A programmer has to manipulate many different classes
- Problem: How can we simplify the view that programmers have of a complex package.
- Force:

  Understanding and using an entire subsystem is hard